COMP 2011 Midterm Exam - Fall 2024 - HKUST

Date: October 26, 2024 (Saturday)
Time Allowed: 2 hours, 10:00-12:00noon
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are 8 questions on **22** pages (including this cover page, honor code, and **4** blank pages for rough work at the end).
3. Write your answers in the space provided in black/blue ink. NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.
4. All programming codes in your answers must be written in the ANSI C++ version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

Student Name	
Student ID	
Email Address	
Lecture & Lab Section	

Problem	**Score**
1	/4
2	/6
3	/8
4	/6
5	/6
6	/10
7	/15
8	/15
Total	/ 70

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Problem 1 [4 points] C++ Operators

What is the output of the following code:

```
#include <iostream>
using namespace std;

int main() {
    short n = 3;
    int m = (n = 4);
    float i = 1.7;

    cout << n++ << endl;
    cout << n << endl;

    cout << --m << endl;
    cout << m << endl;

    cout << (i * m) - (n % m) << endl;
    cout << m / n << endl;

    return 0;
}
```

Answer:

Problem 2 [6 points] Function Call and Return

```
#include <iostream>
using namespace std;

int X(int& a, int& b) {
    return ++a + b--;
}

int& Y(int& c, int d) {
    int& tmp = d;
    d += c;
    c -= tmp;
    return c;
}

int Z(const int& e, int& f) {
    int tmp = e;
    const int& tmp2 = f;
    f += e;
    tmp -= tmp2;
    return tmp;
}

int main() {
    int a=2, b=5;
    const int c=3;

    // This comment is to be replaced by the statements in the question

    return 0;
}
```

The above program compiles without any error. If each of the following statements replaces the comment in the `main()` body, can it still be compiled successfully? If so, please write the output from running the program. Otherwise, please explain the error.

(a) `cout << X(a,b) << " "; cout << Y(b,c) << " "; cout << Z(c,a) << endl;`

(b) `cout << X(Y(a,b),c) << " "; cout << a << " "; cout << b << endl;`

(c) `cout << Z(Y(a,c),Y(b,c)) << " "; cout << a << " "; cout << b << endl;`

Answer:

(a) *(2 points)*

(b) *(2 points)*

(c) *(2 points)*

Problem 3 [8 points] Control

Please rewrite the following program using if/else blocks instead of switch, so that the decisions and output are the same:

```
#include<iostream>
using namespace std;
enum Prize {First, Second, Third};
int main(){
    char result, name;
    int value;

    cin >> name; cin >> value;

    switch(name){
        case 'a':
            switch(value){
                case First:
                    result = 't';
                    break;
                case Second:
                    result = 's';
                case Third:
                    result = 's';
                default:
                    result = 'e';
            }
        case 'c':
            result = 'r';
            break;
        default:
            result = '3';
    }
    return 0;
}
```

Answer:

```
#include<iostream>
using namespace std;
enum Prize {First, Second, Third};

int main(){
    char result, name;
    int value;

    cin >> name; cin >> value;

    /* BEGIN - write your code block - BEGIN */









    /* END - write your code block - END */
    return 0;
}
```

Problem 4 [6 points] Print Pattern

Fill in the blanks in the function `functionv(int)`, as shown below, so that calling `functionv(7)` prints the following pattern.

```
* # # # # # # # # # # # *
# * # # # # # # # # # * #
# # * # # # # # # # * # #
# # # * # # # # # * # # #
# # # # * # # # * # # # #
# # # # # * # * # # # # #
# # # # # # * # # # # # #
```

Answer:

```
#include <iostream>
using namespace std;

void functionv(_____________________){
    int row, pound, star;
    for(row=1; row<= _____________________; row++){

        for(pound=1; pound<= _____________________; pound++)
            cout << "#";
        for(star=1; star<= _____________________; star++)

            if(star == _____________________ || star == _____________________)
                cout << "*";
            else
                cout << "#";
        for(pound=1; pound<= _____________________; pound++)
            cout << "#";

        _____________________;
    }
}

int main(){
    functionv(7);
    return 0;
}
```

Problem 5 [6 points] C String

```
#include <iostream>
using namespace std;

const int N = 26;

bool mystery(const char s1[], const char s2[]) {
   int arr[N] = {};
   int i = 0, j = 0;

   for (; s1[i] != '\0'; ++i);

   for (; s2[j] != '\0'; ++j)
      arr[s2[j] - 'a']++;

   if (i != j)
      return false;

   for (int k = i-1; k >= 0; --k)
      if (arr[s1[k] - 'a'])
         arr[s1[k] - 'a']--;

   for (int k=0; k<N; ++k)
      if (arr[k])
         return false;

   return true;
}

int main() {
   cout << mystery("abc", "acba") << endl; // Part (a)
   cout << mystery("abc", "cba") << endl; // Part (b)
   cout << mystery("tommarvoloriddle", "iamlordvoldemort") << endl; // Part (c)

   return 0;
}
```

Write down the 3 lines of outputs of the program separately in parts (a), (b) and (c).

(a) **Answer:** _____

(b) **Answer:** _____

(c) **Answer:** _____

Problem 6 [10 points] 2D Array

Figure 1 shows a 2D grid (array) of cells, each of which may be either black (1) or white (0) in color. The black cells that are connected form a blob. White cells do not form a blob. Black cells are connected if they are adjacent horizontally, vertically, or diagonally. There may be several blobs on the 2D grid.

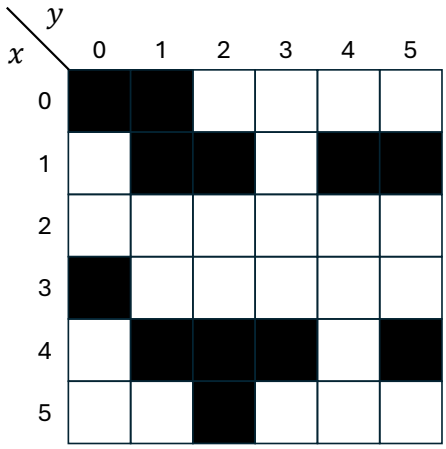


Figure 1: 2D grid (array) of cells.

We use a function called `CountBlob` that accepts as input the 2D grid and the coordinates (x, y) of a particular cell. `CountBlob` returns the size of the block containing the cell. The blob size is defined as the number of black cells in a blob. As shown in Figure 1, if the function arguments represent the $x-$ and $y-$coordinates of a cell, the return value of `CountBlob(1,4)` is 2. The return values of `CountBlob(4,2)`, `CountBlob(4,5)` and `CountBlob(5,5)` are 5, 1 and 0, respectively.

You are asked to write the recursive function `CountBlob`, as shown below (zero marks will be given if the function is not recursive).

```
#include <iostream>
using namespace std;
const int WHITE = 0;
const int BLACK = 1;
const int ROW = 6;
const int COL = 6;
```

```
int CountBlob(int grid[][COL], int x, int y){
    /* BEGIN - write your code block - BEGIN */









    /* END - write your code block - END */
}

int main(){
    int grid[ROW][COL] = {{BLACK, BLACK, WHITE, WHITE, WHITE, WHITE},
        {WHITE, BLACK, BLACK, WHITE, BLACK, BLACK},
        {WHITE, WHITE, WHITE, WHITE, WHITE, WHITE},
        {BLACK, WHITE, WHITE, WHITE, WHITE, WHITE},
        {WHITE, BLACK, BLACK, BLACK, WHITE, BLACK},
        {WHITE, WHITE, BLACK, WHITE, WHITE, WHITE}};
    cout << "Blob size at (1, 4) is " << CountBlob(grid, 1, 4) << endl;
}
```

Problem 7 [15 points] 1D Array

Recall that in the lecture notes of the topic "Array", there is a function which checks whether a C String is a palindrome or not.

In the question, instead of C String, we are checking whether a sequence of positive integers stored in an array is a palindrome or not. A palindrome of integers means that the sequence of the integers reads the same backwards as forwards. For example, the sequence of integers, 1, 3, 3, 1, is a palindrome.

Besides checking a palindrome in its basic form with a function, we are also checking a variation of a palindrome with another function.

You are required to write those 2 functions with the given function headers. Below is an example program that demonstrates how they work. Your solution should work for any integer arrays with any number of elements.

Note:

- You can assume all integer values in the integer arrays are positive (i.e. larger than 0).
- You are not allowed to define or create any additional arrays and helper function(s), nor include additional libraries.
- A function required by a later part may call functions written for earlier parts.

```
#include <iostream>
using namespace std;

void print_arr(const int arr[], int size) {
   for (int i=0; i<size; ++i)
      cout << arr[i] << ((i == size-1) ? "\n" : ", ");
}

// Definition of the 2 functions go here ..

int main() {
   int arr1[] = {1, 3, 5, 7, 5, 3, 1};
   int n = sizeof(arr1)/sizeof(int);
   print_arr(arr1, n);
   cout << boolalpha;
   cout << "Is palindrome? " << is_palindrome(arr1, n) << endl;
   cout << "Is rotated palindrome? " << is_rotated_palindrome(arr1, n) << endl;

   int arr2[] = {5, 7, 5, 3, 3};
   n = sizeof(arr2)/sizeof(int);
   print_arr(arr2, n);
   cout << "Is palindrome? " << is_palindrome(arr2, n) << endl;
   cout << "Is rotated palindrome? " << is_rotated_palindrome(arr2, n) << endl;
```

```
    int arr3[] = {5, 7, 5, 3, 5};
    n = sizeof(arr3)/sizeof(int);
    print_arr(arr3, n);
    cout << "Is palindrome? " << is_palindrome(arr3, n) << endl;
    cout << "Is rotated palindrome? " << is_rotated_palindrome(arr3, n) << endl;

    int arr4[] = {1, 6, 6, 1, 4, 4};
    n = sizeof(arr4)/sizeof(int);
    print_arr(arr4, n);
    cout << "Is palindrome? " << is_palindrome(arr4, n) << endl;
    cout << "Is rotated palindrome? " << is_rotated_palindrome(arr4, n) << endl;

    return 0;
}
```

A sample run of the program is shown below.

```
1, 3, 5, 7, 5, 3, 1
Is palindrome? true
Is rotated palindrome? true
5, 7, 5, 3, 3
Is palindrome? false
Is rotated palindrome? true
5, 7, 5, 3, 5
Is palindrome? false
Is rotated palindrome? false
1, 6, 6, 1, 4, 4
Is palindrome? false
Is rotated palindrome? true
```

(a) *(5 points)* Implement the function `is_palindrome()` which takes an integer array, `arr`, with the number of elements, `size`, as input, checks if the sequence of the integer values is a palindrome, and returns `true` if it is, otherwise `false`.

```
bool is_palindrome(const int arr[], int size) {
   // Add your code here









}
```

(b) *(10 points)* Implement the function `is_rotated_palindrome()` which takes an integer array, `arr`, with the number of elements, `size`, as input, and checks whether the sequence after some circular rotations is a palindrome. A circular rotation means shifting forward the elements, where the first element will become the last element in the rotated array. The function returns `true` if the sequence of integer values is a rotated palindrome, otherwise `false`.

```
bool is_rotated_palindrome(const int arr[], int size) {
   // Add your code here









}
```

Problem 8 [15 points] Data Encryption

Given a plain text stored in C-string array `plaintext`, you are to implement two encryption methods as functions that return the encrypted text in C-string array `ciphertext`.

Please assume that `plaintext` contains only lower-case English characters (also known as alphabets or letters) and white spaces (i.e., `' '`). Note that you only need to encrypt alphabet characters and white space in `plaintext` should be left as white space in `ciphertext`. All arrays have sufficient size.

(a) *(6 points) Caesar cipher:* In Caesar cipher, each letter in `plaintext` is shifted by a certain number `shift` forward in a circular manner. For example, if `shift = 3`, `'a'` is encrypted as `'d'`, `'b'` as `'e'`, ...,`'x'` as `'a'`, and `'y'` as `'b'`, and `'z'` as `'c'`.

Implement below Caesar cipher as the `caesarCipher` function, which returns the encrypted text in `ciphertext`.

```
void caesarCipher(const char plaintext[], char ciphertext[], int shift) {
```

(b) *(9 points) Vigenere cipher:* Vigenere cipher is similar to Caesar cipher, but the shift (in a circular manner) of an alphabet in `plaintext` depends on the letter in a C-string key `key`, with `'a'` meaning a shift of 0, `'b'` meaning 1, `'c'` meaning 2, ..., and `'z'` meaning 25. If `key` is shorter than `plaintext`, the reading of letters in `key` would be wrapped around. Note that the key contains only lowercase letters. Because white space is not encrypted, the letter in the `key` is not advanced when encountering a space in `plaintext`.

For example, if plaintext is `"hi there"` and the `key` is `"scrt"`, then `'h'` is encrypted with a shift of 18 (derived from `'s'`) to `'z'`, `'i'` with shift 2 (due to `'c'`) to `'k'`, `' '` with shift 0 (no shift for space), `'t'` with shift 17 (due to `'r'`) to `'k'`, `'h'` with shift 19 (due to `'t'`) to `'a'`, `'e'` with shift 18 (wrapped around to `'s'`) to `'w'`, and so on. The resultant `ciphertext` is hence `"zk kawtv"`.

Implement the Vigenere Cipher as the `vigenereCipher` function, which returns the encrypted text in `ciphertext` given the encryption key `key`:

```
void vigenereCipher(const char plaintext[], char ciphertext[],
            const char key[]) {
```

---------- END OF PAPER ----------

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */